DBMS – ADVANCED SQL QUERY SYSTEM

Project Title: IKEA E-Commerce Database Management System
Task: Advanced SQL Query Implementation
Subject: Database Management Systems

Objectives

1. Implement subqueries and nested queries.
2. Find products above average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare an advanced SQL report.

1. SUBQUERIES AND NESTED QUERIES

A subquery is a query written inside another query. The inner query runs first and gives a value or a list of values, and the outer query uses that result in its WHERE or HAVING condition.

1.1 Single-row Subquery:

A single-row subquery returns one value. Here the inner query finds the highest price and the outer query shows the product with that price.

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price = (SELECT MAX(Price) FROM Product);
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price = (SELECT MAX(Price) FROM Product);

PRODUCT_ID PRODUCT_NAME   PRICE
---------- -------------- -----
       102 MALM Bed Frame 12999

1 row selected.

SQL>
```

1.2 Multiple-row Subquery

A multiple-row subquery returns a list, so IN is used. This shows the products bought by customer 1001, which is only a sample Customer_ID.

```
SELECT Product_ID, Product_Name
FROM Product
WHERE Product_ID IN
(SELECT oi.Product_ID
FROM Order_Item oi
JOIN "Order" o
ON oi.Order_ID = o.Order_ID
WHERE o.Customer_ID = 1001);
```

```
SQL> SELECT Product_ID, Product_Name
  2  FROM Product
  3  WHERE Product_ID IN
  4  (SELECT oi.Product_ID
  5  FROM Order_Item oi
  6  JOIN "Order" o
  7  ON oi.Order_ID = o.Order_ID
  8  WHERE o.Customer_ID = 1001);

PRODUCT_ID PRODUCT_NAME
---------- --------------------
       101 KALLAX Shelf Unit
       103 LACK Coffee Table
       109 APPLARO Outdoor Table

3 rows selected.

SQL>
```

1.3 Nested Subquery

In a nested query, one subquery is placed inside another. The innermost query finds the average price, the middle query finds the categories of products priced above it, and the outer query lists all products in those categories.

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE Category_ID IN
(SELECT Category_ID FROM Product
WHERE Price >
(SELECT AVG(Price) FROM Product));
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE Category_ID IN
  4  (SELECT Category_ID FROM Product
  5  WHERE Price >
  6  (SELECT AVG(Price) FROM Product));

PRODUCT_ID PRODUCT_NAME          CATEGORY_ID
---------- -------------------- -----------
       102 MALM Bed Frame                 2
       103 LACK Coffee Table              1
       104 POANG Armchair                 1
       105 MICKE Desk                     5
       109 APPLARO Outdoor Table          6

5 rows selected.

SQL>
```

1.4 Correlated Subquery

A correlated subquery uses a value from the outer query, so it runs once for every outer row. This shows products that cost more than the average price of their own category.

```
SELECT p.Product_ID, p.Product_Name,
p.Category_ID, p.Price
FROM Product p
WHERE p.Price >
(SELECT AVG(p2.Price)
FROM Product p2
WHERE p2.Category_ID = p.Category_ID);
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  p.Category_ID, p.Price
  3  FROM Product p
  4  WHERE p.Price >
  5  (SELECT AVG(p2.Price)
  6  FROM Product p2
  7  WHERE p2.Category_ID = p.Category_ID);

PRODUCT_ID PRODUCT_NAME      CATEGORY_ID PRICE
---------- ---------------- ----------- -----
       101 KALLAX Shelf Unit           7  4999
       104 POANG Armchair              1  6999

2 rows selected.

SQL>
```

2. PRODUCTS ABOVE AVERAGE PRICE

The AVG() function gives the average price of all IKEA products.

```
SELECT ROUND(AVG(Price), 2) AS Average_Price
FROM Product;
```

```
SQL> SELECT ROUND(AVG(Price), 2) AS Average_Price
  2  FROM Product;

AVERAGE_PRICE
-------------
         5079

1 row selected.

SQL>
```

2.1 Products Above Average Price

The average is calculated by a subquery, so the condition keeps working when prices change.

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price > (SELECT AVG(Price) FROM Product)
ORDER BY Price DESC;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > (SELECT AVG(Price) FROM Product)
  4  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME          PRICE
---------- -------------------- -----
       102 MALM Bed Frame       12999
       109 APPLARO Outdoor Table  8999
       104 POANG Armchair         6999
       105 MICKE Desk             5999

4 rows selected.

SQL>
```

2.2 Products Below Average Price

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price < (SELECT AVG(Price) FROM Product)
ORDER BY Price DESC;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < (SELECT AVG(Price) FROM Product)
  4  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME           PRICE
---------- ---------------------- -----
       101 KALLAX Shelf Unit       4999
       106 INGOLF Dining Chair     3999
       103 LACK Coffee Table       2499
       108 MULIG Clothes Rack      1999
       107 FADO Table Lamp         1499
       110 FEJKA Artificial Plant   799

6 rows selected.

SQL>
```

3. CUSTOMERS WITH MAXIMUM PURCHASES

First the total purchase amount of each customer is calculated, and then the maximum of these totals is found with MAX() and a subquery.

3.1 Customer-wise Purchase Amount

```
SELECT c.Customer_ID, c.Customer_Name,
SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Purchase DESC;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  SUM(o.Total_Amount) AS Total_Purchase
  3  FROM Customer c
  4  JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME TOTAL_PURCHASE
----------- ------------ --------------
       1004 Meena Priya           22296
       1001 Arun Kumar            18996
       1005 Suresh Babu           18393
       1002 Divya Lakshmi         16096
       1003 Karthik Raja           9997
       1006 Anitha Devi            6498

6 rows selected.

SQL>
```

3.2 Maximum Purchase Amount

```
SELECT MAX(Total_Purchase) AS Maximum_Purchase
FROM (SELECT SUM(Total_Amount) AS Total_Purchase
FROM "Order"
GROUP BY Customer_ID);
```

```
SQL> SELECT MAX(Total_Purchase) AS Maximum_Purchase
  2  FROM (SELECT SUM(Total_Amount) AS Total_Purchase
  3  FROM "Order"
  4  GROUP BY Customer_ID);

MAXIMUM_PURCHASE
----------------
           22296

1 row selected.

SQL>
```

3.3 Customer with the Maximum Purchase

HAVING compares each customer total with the maximum. Because the comparison uses =, every customer who shares the highest amount is displayed.

```
SELECT c.Customer_ID, c.Customer_Name,
SUM(o.Total_Amount) AS Total_Purchase
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
HAVING SUM(o.Total_Amount) =
(SELECT MAX(Total_Purchase)
FROM (SELECT SUM(Total_Amount) AS Total_Purchase
FROM "Order"
GROUP BY Customer_ID));
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  SUM(o.Total_Amount) AS Total_Purchase
  3  FROM Customer c
  4  JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  HAVING SUM(o.Total_Amount) =
  8  (SELECT MAX(Total_Purchase)
  9  FROM (SELECT SUM(Total_Amount) AS Total_Purchase
 10  FROM "Order"
 11  GROUP BY Customer_ID));

CUSTOMER_ID CUSTOMER_NAME TOTAL_PURCHASE
----------- ------------- --------------
       1004 Meena Priya           22296

1 row selected.

SQL>
```

4. COMPLEX BUSINESS QUERIES

4.1 Orders Above the Average Order Value

This lists the customers and their orders whose amount is higher than the average order value.

```
SELECT c.Customer_ID, c.Customer_Name,
o.Order_ID, o.Total_Amount
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
WHERE o.Total_Amount >
(SELECT AVG(Total_Amount) FROM "Order")
ORDER BY o.Total_Amount DESC;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  o.Order_ID, o.Total_Amount
  3  FROM Customer c
  4  JOIN "Order" o
  5  ON c.Customer_ID = o.Customer_ID
  6  WHERE o.Total_Amount >
  7  (SELECT AVG(Total_Amount) FROM "Order")
  8  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID TOTAL_AMOUNT
----------- ------------- -------- ------------
       1005 Suresh Babu       5006        18393
       1004 Meena Priya       5009        12499
       1002 Divya Lakshmi     5002        11999

3 rows selected.

SQL>
```

4.2 Products with Sales Above the Average Product Sales

The inner query finds the total sales of every product, and the average of those totals is used to filter the products.

```
SELECT p.Product_ID, p.Product_Name,
SUM(oi.Total_Price) AS Total_Sales
FROM Product p
JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
HAVING SUM(oi.Total_Price) >
(SELECT AVG(Product_Sales)
FROM (SELECT SUM(Total_Price) AS Product_Sales
FROM Order_Item
GROUP BY Product_ID))
ORDER BY Total_Sales DESC;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  SUM(oi.Total_Price) AS Total_Sales
  3  FROM Product p
  4  JOIN Order_Item oi
  5  ON p.Product_ID = oi.Product_ID
  6  GROUP BY p.Product_ID, p.Product_Name
  7  HAVING SUM(oi.Total_Price) >
  8  (SELECT AVG(Product_Sales)
  9  FROM (SELECT SUM(Total_Price) AS Product_Sales
 10  FROM Order_Item
 11  GROUP BY Product_ID))
 12  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME        TOTAL_SALES
---------- ------------------ -----------
       102 MALM Bed Frame            24498
       106 INGOLF Dining Chair       15996
       101 KALLAX Shelf Unit          9998

3 rows selected.

SQL>
```

4.3 Stock Availability Across Warehouses

This shows the products that are in stock in more than one warehouse, with the stock in each warehouse.

```
SELECT p.Product_Name, w.Warehouse_Name,
i.Stock_Quantity
FROM Product p
JOIN Inventory i
ON p.Product_ID = i.Product_ID
JOIN Warehouse w
ON i.Warehouse_ID = w.Warehouse_ID
WHERE i.Stock_Quantity > 0
AND p.Product_ID IN
(SELECT Product_ID FROM Inventory
WHERE Stock_Quantity > 0
GROUP BY Product_ID
HAVING COUNT(*) > 1)
ORDER BY p.Product_ID, w.Warehouse_ID;
```

```
SQL> SELECT p.Product_Name, w.Warehouse_Name,
  2  i.Stock_Quantity
  3  FROM Product p
  4  JOIN Inventory i
  5  ON p.Product_ID = i.Product_ID
  6  JOIN Warehouse w
  7  ON i.Warehouse_ID = w.Warehouse_ID
  8  WHERE i.Stock_Quantity > 0
  9  AND p.Product_ID IN
 10  (SELECT Product_ID FROM Inventory
 11  WHERE Stock_Quantity > 0
 12  GROUP BY Product_ID
 13  HAVING COUNT(*) > 1)
 14  ORDER BY p.Product_ID, w.Warehouse_ID;

PRODUCT_NAME      WAREHOUSE_NAME    STOCK_QUANTITY
----------------- ----------------- --------------
KALLAX Shelf Unit Chennai Warehouse             15
KALLAX Shelf Unit Mumbai Warehouse              10
LACK Coffee Table Chennai Warehouse             12
LACK Coffee Table Mumbai Warehouse               8
MICKE Desk        Chennai Warehouse              5
MICKE Desk        Mumbai Warehouse              10
FADO Table Lamp   Chennai Warehouse             25
FADO Table Lamp   Mumbai Warehouse              15

8 rows selected.

SQL>
```

4.4 Products and Their Suppliers

This lists the products of suppliers who supply more than two products, together with the supplier name.

```
SELECT p.Product_ID, p.Product_Name, p.Price,
s.Supplier_Name
FROM Product p
JOIN Supplier s
ON p.Supplier_ID = s.Supplier_ID
WHERE p.Supplier_ID IN
(SELECT Supplier_ID FROM Product
GROUP BY Supplier_ID
HAVING COUNT(*) > 2)
ORDER BY s.Supplier_Name, p.Product_ID;
```

```
SQL> SELECT p.Product_ID, p.Product_Name, p.Price,
  2  s.Supplier_Name
  3  FROM Product p
  4  JOIN Supplier s
  5  ON p.Supplier_ID = s.Supplier_ID
  6  WHERE p.Supplier_ID IN
  7  (SELECT Supplier_ID FROM Product
  8  GROUP BY Supplier_ID
  9  HAVING COUNT(*) > 2)
 10  ORDER BY s.Supplier_Name, p.Product_ID;

PRODUCT_ID PRODUCT_NAME        PRICE SUPPLIER_NAME
---------- ------------------ ----- ------------------
       103 LACK Coffee Table    2499 HomeLine Traders
       105 MICKE Desk           5999 HomeLine Traders
       108 MULIG Clothes Rack   1999 HomeLine Traders
       101 KALLAX Shelf Unit    4999 Woodcraft Supplies
       102 MALM Bed Frame      12999 Woodcraft Supplies
       106 INGOLF Dining Chair  3999 Woodcraft Supplies

6 rows selected.

SQL>
```

5. CUSTOMER PURCHASE REPORT

This report shows the number of orders, the total purchase amount and the average order value of every customer.

```
SELECT c.Customer_ID, c.Customer_Name,
COUNT(o.Order_ID) AS Total_Orders,
SUM(o.Total_Amount) AS Total_Purchase,
ROUND(AVG(o.Total_Amount), 2) AS Avg_Order_Value
FROM Customer c
JOIN "Order" o
ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Purchase DESC;
```

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
  2  COUNT(o.Order_ID) AS Total_Orders,
  3  SUM(o.Total_Amount) AS Total_Purchase,
  4  ROUND(AVG(o.Total_Amount), 2) AS Avg_Order_Value
  5  FROM Customer c
  6  JOIN "Order" o
  7  ON c.Customer_ID = o.Customer_ID
  8  GROUP BY c.Customer_ID, c.Customer_Name
  9  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME TOTAL_ORDERS TOTAL_PURCHASE AVG_ORDER_VALUE
----------- ------------ ------------ -------------- ---------------
       1004 Meena Priya             2          22296           11148
       1001 Arun Kumar              2          18996            9498
       1005 Suresh Babu             1          18393           18393
       1002 Divya Lakshmi           2          16096            8048
       1003 Karthik Raja            1           9997            9997
       1006 Anitha Devi             1           6498            6498

6 rows selected.

SQL>
```

6. ADVANCED PRODUCT SALES REPORT

The report uses Total_Price from Order_Item, which holds the amount charged after discount. The average selling price is the total sales divided by the total quantity sold.

```
SELECT p.Product_ID, p.Product_Name,
COUNT(DISTINCT oi.Order_ID) AS Order_Count,
SUM(oi.Quantity) AS Total_Qty,
SUM(oi.Total_Price) AS Total_Sales,
ROUND(SUM(oi.Total_Price) / SUM(oi.Quantity), 2)
AS Avg_Price
FROM Product p
JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Product_ID, p.Product_Name
ORDER BY Total_Sales DESC;
```

```
SQL> SELECT p.Product_ID, p.Product_Name,
  2  COUNT(DISTINCT oi.Order_ID) AS Order_Count,
  3  SUM(oi.Quantity) AS Total_Qty,
  4  SUM(oi.Total_Price) AS Total_Sales,
  5  ROUND(SUM(oi.Total_Price) / SUM(oi.Quantity), 2)
  6  AS Avg_Price
  7  FROM Product p
  8  JOIN Order_Item oi
  9  ON p.Product_ID = oi.Product_ID
 10  GROUP BY p.Product_ID, p.Product_Name
 11  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME           ORDER_COUNT TOTAL_QTY TOTAL_SALES AVG_PRICE
---------- ---------------------- ----------- --------- ----------- ---------
       102 MALM Bed Frame                   2         2       24498     12249
       106 INGOLF Dining Chair              1         4       15996      3999
       101 KALLAX Shelf Unit                2         2        9998      4999
       109 APPLARO Outdoor Table            1         1        8999      8999
       103 LACK Coffee Table                2         3        7497      2499
       104 POANG Armchair                   1         1        6999      6999
       105 MICKE Desk                       1         1        5999      5999
       107 FADO Table Lamp                  2         3        4297      1432
       108 MULIG Clothes Rack               1         2        3998      1999
       110 FEJKA Artificial Plant           2         5        3995       799

10 rows selected.

SQL>
```

7. CATEGORY-WISE ADVANCED SALES REPORT

Product and Order_Item are joined to get the sales of each category. The category name column of the Category table is not confirmed, so Category_ID is shown. If the column exists, join Category on Category_ID and select its name as well.

```
SELECT p.Category_ID,
COUNT(DISTINCT p.Product_ID) AS Product_Count,
NVL(SUM(oi.Quantity), 0) AS Total_Qty,
NVL(SUM(oi.Total_Price), 0) AS Total_Sales
FROM Product p
LEFT JOIN Order_Item oi
ON p.Product_ID = oi.Product_ID
GROUP BY p.Category_ID
ORDER BY Total_Sales DESC;
```

```
SQL> SELECT p.Category_ID,
  2  COUNT(DISTINCT p.Product_ID) AS Product_Count,
  3  NVL(SUM(oi.Quantity), 0) AS Total_Qty,
  4  NVL(SUM(oi.Total_Price), 0) AS Total_Sales
  5  FROM Product p
  6  LEFT JOIN Order_Item oi
  7  ON p.Product_ID = oi.Product_ID
  8  GROUP BY p.Category_ID
  9  ORDER BY Total_Sales DESC;

CATEGORY_ID PRODUCT_COUNT TOTAL_QTY TOTAL_SALES
----------- ------------- --------- -----------
          2             1         2       24498
          4             1         4       15996
          1             2         4       14496
          7             2         4       13996
          6             1         1        8999
          5             1         1        5999
          8             1         3        4297
         10             1         5        3995

8 rows selected.

SQL>
```

8. CONCLUSION

In this task, subqueries and nested queries were used on the IKEA E-Commerce Database to compare products with the average price, find the customers with the highest purchases, and answer business questions about orders, product sales, warehouse stock and suppliers. The customer, product and category reports combine joins, aggregate functions and subqueries, which helps in analysing product prices, customer purchases, inventory and sales performance.